PLAN DE PROYECTO / VERSIÓN 1.1

PowerZone

Plan integral de desarrollo: del 0 al 100 %

Ramas, Conventional Commits, reparto de trabajo, sprints, calidad y entrega.

Equipo de tres Alejandro Alzate · Juan Diego Herrera · Juan José Guarumo

Base de este plan

- **Notion:** requisitos del sistema con 14 módulos funcionales, 40 requisitos funcionales y 22 no funcionales; actas y documentos de análisis.

- **Jira:** proyecto PPP con 16 épicas, 40 historias, 243 subtareas y 4 tareas transversales. Hay 13 sprints configurados entre octubre de 2026 y abril de 2027.

- **Repositorio:** Angular 22, Spring Boot 3.5/Java 21, PostgreSQL 16 y FastAPI/Python 3.12; Docker Compose y CI.

Cómo leerlo El porcentaje 0-100 describe hitos de entrega, no el avance verificado hoy. Jira mantiene 303 incidencias 'por hacer', aunque Git contiene implementación. Se asignaron responsables a los 299 tickets originales y a 4 tareas nuevas; la carga debe revisarse por estimación, no por número de tickets.

Corte de información: 22 de octubre de 2026 (hora de Bogotá).

Fuentes principales: requisitos y actas en Notion, backlog PPP en Jira, README, arquitectura, CI y Git del repositorio.

1. Objetivo, alcance y arquitectura

Centralizar la gestión de gimnasios y la experiencia del cliente: usuarios, membresías y pagos, rutinas, clases, asistencia, inventario, nutrición, comunicación, reportes y asistencia inteligente. La entrega final debe cumplir criterios funcionales y no funcionales, además de ser operable, documentada y recuperable.

Capa	Decisión actual	Regla de trabajo
Web	Angular 22 + PWA	Una ruta y servicio por dominio; roles y estados en UI; diseño móvil y accesible.
API	Spring Boot 3.5 / Java 21	Módulos por dominio; controlador, servicio y persistencia; contrato OpenAPI estable.
Datos	PostgreSQL 16 + Flyway + JDBC	Migraciones nuevas e irreversibles; conservar datos y aislamiento por gimnasio.
IA	FastAPI / Python 3.12	Integración privada por backend; fallback; evaluar modelo real y privacidad antes de ampliarlo.
Operación	Docker Compose + GitHub Actions	Configurar entornos, secretos, salud, copias, monitoreo y despliegue reproducible.

Alcance funcional

Los 14 módulos de la especificación y de Jira son: usuarios/rutinas, pagos/membresías, guías de ejercicio, administración multi-gimnasio, creador de rutinas, hábitos, nutrición, inventario, notificaciones, asistente IA, acceso físico, clases/reservas, marketing y reportes.

Criterios no funcionales que requieren evidencia

Usabilidad y diseño responsivo; disponibilidad 99,8 %; recuperación en 5 minutos; control de acceso y auditoría; mayoría de operaciones en menos de 2 segundos; respaldo diario; compatibilidad web/móvil y documentación. Son objetivos de la especificación, no resultados medidos. Cada uno necesita prueba, responsable y evidencia antes del cierre.

Decisión técnica

La documentación antigua menciona Node.js/NestJS y algunas tareas nuevas proponen JPA. El runtime actual usa Spring Boot con JDBC/TableStore. Mantener esta arquitectura como referencia hasta que una decisión formal cambie el diseño; actualizar los requisitos y tickets para evitar implementaciones incompatibles.

2. Ruta de entrega: del 0 al 100 %

Los rangos son puertas de salida acumulativas. No se asigna porcentaje por número de commits ni por horas invertidas: una fase se valida con entregables y evidencias.

Tramo	Entregable verificable	Cierre de fase
0-10 %	Problema, usuarios, objetivos, alcance, ERS, priorización y criterios de aceptación.	Notion aprobado; 40 historias trazadas en Jira.
10-20 %	Repositorio, arquitectura, entornos, esquema inicial, CI, seguridad base y diseño UX.	Instalación reproducible; migración y CI pasan.
20-40 %	Identidad, roles, gimnasios, usuarios, ejercicios, rutinas y panel inicial.	Flujos de alta, login y rutinas probados por rol y sede.
40-60 %	Membresías, pagos, clases, reservas, asistencia, nutrición e inventario.	Operaciones transaccionales y casos de error validados.
60-75 %	Comunicaciones, marketing, reportes, hábitos, guías, IA y voz según prioridad aprobada.	Historias de valor y límites de IA demostrados.
75-90 %	Integración, pruebas, seguridad, rendimiento, accesibilidad, observabilidad y respaldos.	CI y pruebas E2E; restauración y objetivos no funcionales medidos.
90-100 %	Aceptación, manuales, release, despliegue, monitoreo, soporte y retrospectiva.	Acta de aceptación; versión etiquetada y plan de reversión.

Trazabilidad mínima por cada historia

Requisito Notion -> épica/historia Jira -> rama y PR -> commits -> prueba -> evidencia de aceptación -> versión. El vínculo debe figurar en la descripción de Jira y en el PR. Las incidencias sin criterio de aceptación o dueño no entran al sprint.

Qué significa terminar

El 100 % exige las funciones comprometidas con el alcance final, no solo una compilación correcta. Si se aplaza un requisito opcional, debe registrarse la decisión, la versión destino y la aprobación del responsable de producto.

Hitos históricos y planificados

Agosto-septiembre de 2025: análisis, actas iniciales y ERS en Notion. 27 de agosto de 2026: primer commit y módulos iniciales. 5-8 de octubre de 2026: migración a Spring Boot/FastAPI. 19 de octubre de 2026 a 21 de abril de 2027: sprints de desarrollo registrados en Jira; sus fechas requieren validación de capacidad.

3. Cronograma registrado en Jira

Fechas de los sprints del tablero PPP. Las cifras cuentan historias y subtareas asignadas al sprint; no equivalen a esfuerzo validado ni a capacidad disponible.

Sprint	Foco	Ventana registrada	Items
0	Foundation	5-17 oct 2026	0
1	Auth y usuarios	19-31 oct 2026	29
2	Rutinas	2-14 nov 2026	22
3	Pagos	16-28 nov 2026	21
4	Administración y CI	30 nov-12 dic 2026	11
5	Biblioteca y creador	29 ene-13 feb 2027	11
6	Tracking	15-28 feb 2027	9
7	Acceso físico	1-14 mar 2027	18
8	Clases y reservas	15-28 mar 2027	20
9	Notificaciones y nutrición	29 mar-14 abr 2027	28
10	Inventario y guías	5-11 abr 2027	25
11	IA y marketing	12-14 abr 2027	49
12	Reportes y cierre	16 mar-21 abr 2027	40

Replanificación

Sprints 9-12 se solapan. El sprint 11 concentra 49 items en tres días y el 10, 25 en siete días; el 12 dura más de un mes. Hay un hueco entre el 12 de diciembre y el 29 de enero. Reestimar con capacidad real de tres personas y fijar una fecha de entrega aprobada antes de tratar el 21 de abril como compromiso.

Cadencia recomendada

Planificación y objetivo por sprint; reunión breve diaria; refinamiento semanal; demo y retrospectiva al cierre. Limitar trabajo en curso a una historia principal por persona y reservar capacidad para defectos, revisión y documentación.

4. Equipo de tres: distribución aplicada

La documentación histórica registra a Juan Diego como Scrum Master/backend, a Alejandro en pruebas y a Juan José en frontend. La siguiente distribución adapta esas funciones al equipo de tres solicitado. El rol de Product Owner y la autoridad de aceptación deben confirmarse con el equipo.

Integrante	Responsabilidad principal	Co-responsabilidad
Alejandro Alzate	Coordinación técnica y calidad; arquitectura, seguridad, CI y release.	Administración, inventario, marketing y reportes; pruebas de aceptación.
Juan Diego Herrera	Scrum y backend; dominio, integración y contrato API.	Pagos, membresías, rutinas, nutrición, acceso, clases e IA.
Juan José Guarumo	Frontend y experiencia; Angular/PWA, flujos, accesibilidad y diseño.	Guías, hábitos, comunicaciones, gráficos y voz en UI.

Dueño principal por épica de Jira

Alejandro: PPP-1 Infraestructura; PPP-5 administración; PPP-9 inventario; PPP-14 marketing; PPP-15 reportes; PPP-16 calidad/DevOps.

Juan Diego: PPP-2 usuarios/rutinas; PPP-3 pagos; PPP-6 creador de rutinas; PPP-8 nutrición; PPP-11 IA; PPP-12 acceso físico; PPP-13 clases.

Juan José: PPP-4 guías; PPP-7 hábitos; PPP-10 comunicaciones.

Distribución aplicada: Alejandro 106 tickets originales, Juan Diego 134 y Juan José 59; además, 3 tareas transversales para Alejandro y 1 para Juan Diego. El volumen de tickets no representa horas ni complejidad.

La propiedad de épica coordina alcance y aceptación técnica, pero no impide que otro integrante implemente partes de la historia. Cada ticket tendrá un único responsable de ejecución y un revisor distinto.

Revisión cruzada

Alejandro revisa entregas de Juan Diego; Juan Diego revisa API e integración de Alejandro/Juan José; Juan José revisa UX de los otros dos. Cambios de seguridad, pagos, migraciones y despliegue requieren dos miradas y evidencia de prueba.

5. Estructura de ramas y flujo de integración

Mantener dos ramas estables porque el repositorio ya usa main y develop. Trabajar por ticket en ramas cortas y eliminar la rama después del merge; no crear ramas permanentes por persona.

Rama	Uso y origen	Destino
main	Código liberado; solo recibe PR de release/hotfix; etiqueta SemVer.	Producción / demostración final
develop	Integración del siguiente release; rama base de historias.	PR a main al cerrar release
feat/PPP-123-resumen	Nueva función desde develop. Ej.: feat/PPP-18-refresh-token.	PR a develop
fix/PPP-123-resumen	Corrección no urgente desde develop.	PR a develop
docs/PPP-123-resumen	Documentación; igual para test/, refactor/, chore/.	PR a develop
hotfix/PPP-123-resumen	Fallo crítico desde main; exige prueba de regresión.	PR a main y sincronizar develop
release/vX.Y.Z	Congelar cambios y validar versión cuando haga falta.	PR a main; luego tag

Secuencia obligatoria

Jira listo -> rama desde develop -> commits pequeños -> PR vinculado -> CI + revisión -> squash merge a develop -> demo/aceptación -> release PR a main -> tag y notas. En hotfix: main -> corrección -> main -> portar a develop.

Protecciones recomendadas

Bloquear push directo y force-push en main/develop; exigir al menos un revisor no autor, CI verde y conversaciones resueltas; invalidar aprobaciones al cambiar código; restringir secretos y exigir que el PR incluya ticket, alcance, pruebas y capturas si cambia UI.

Rama actual feat/spring-boot-fastapi contiene la migración Spring/FastAPI y sigue separada de main en el checkout analizado. Antes de planear entregas sobre main, revisar diferencias y abrir el PR de integración sin reescribir la historia compartida.

6. Conventional Commits y versionado

Formato: **`tipo(alcance): descripción breve`**. Usar tipo y alcance en minúsculas, verbo concreto y un solo propósito por commit. Agregar el identificador Jira en el cuerpo o al final del asunto cuando mejore la trazabilidad.

Tipo	Cuándo	Ejemplo
feat	Función observable	feat(auth): renueva el token al expirar (PPP-18)
fix	Defecto	fix(payments): evita duplicar cobros en webhook (PPP-22)
docs	Documentación	docs(api): actualiza ejemplos de pagos (PPP-21)
test	Pruebas	test(attendance): cubre acceso vencido (PPP-44)
refactor	Estructura sin cambio funcional	refactor(routines): separa validación del servicio
chore	Herramientas/dependencias	chore(ci): agrega análisis de dependencias
perf / build / ci	Rendimiento o infraestructura	perf(reports): reduce tiempo de consulta
revert	Revertir commit	revert: feat(auth): renueva el token

Alcances sugeridos

auth, users, gyms, routines, exercises, memberships, payments, classes, attendance, nutrition, inventory, notifications, marketing, reports, ai, frontend, api, db, ci, docs. Evitar scopes personales o ambiguos como 'misc'.

Cambios incompatibles y versiones

Usar `feat(api)!: cambia el contrato de sesión` y un pie `BREAKING CHANGE:` con impacto y migración. Aplicar SemVer: PATCH para fix, MINOR para feat compatible y MAJOR para ruptura. Etiquetar solo commits de main con `vX.Y.Z`; publicar notas de release con tickets y pasos de migración.

Higiene de commit y PR

Un commit debe compilar y explicar una decisión. No incluir claves, .env, datos reales ni binarios generados. PR idealmente de una historia; si cambia varios dominios, dividirlo. El título del PR seguirá Conventional Commits porque el squash lo convierte en el commit de integración.

7. Jira, aceptación y colaboración

Estructura: épica = resultado de módulo; historia = valor verificable para un rol; subtarea = trabajo técnico. Vincular Notion, PR y evidencia a cada historia. PPP-21, registro de pagos, fue corregida y ahora pertenece a PPP-3.

Estado recomendado	Entrada / salida
Backlog	Requisito identificado; prioridad preliminar.
Listo	Historia pequeña, criterios Given/When/Then, diseño/contrato, dependencias y estimación acordados.
En curso	Responsable asignado; rama creada; avance y bloqueos visibles.
En revisión	PR abierto; CI verde; revisión por compañero; sin cambios pendientes.
En validación	Demo, pruebas y aceptación funcional documentadas.
Hecho	Fusionado, desplegable, documentación y evidencia enlazadas.

Definition of Ready

Objetivo de usuario, alcance, criterios de aceptación, diseño/contrato, dependencias, prioridad, estimación y responsable. Si afecta datos o permisos, incluir plan de migración y casos de error.

Definition of Done

Código revisado; compilación y pruebas adecuadas al riesgo; contrato OpenAPI y migración actualizados; controles de autorización por rol/gimnasio; logs sin datos sensibles; documentación y capturas; aprobación funcional; ticket y PR vinculados.

Plantilla mínima de PR

Jira | problema y solución | cambios de API/BD | riesgos y reversión | pruebas ejecutadas | evidencia de UI | checklist de seguridad | revisor. El autor resuelve comentarios y no se autoaprueba.

8. Calidad, seguridad y salida a producción

La CI actual ejecuta verificación de contrato, Maven, build Angular y un smoke de API con PostgreSQL/FastAPI. La rama activa no contiene pruebas unitarias del backend ni del frontend; el plan debe añadirlas sobre reglas de negocio y flujos críticos, evitando tests que solo repitan implementación.

Puerta	Evidencia requerida
Por PR	Lint/formato; build; pruebas unitarias relevantes; contrato; revisión humana; análisis de dependencias.
Por sprint	Pruebas integración con PostgreSQL; escenarios de rol y gimnasio; demo de historias aceptadas.
Pre-release	E2E de registro, login, pago, reserva, check-in, rutina y reportes; accesibilidad/responsivo.
Operación	Backups diarios y restauración probada; monitoreo, alertas, métricas y procedimientos de incidente.
Entrega	Manual de usuario y técnico; OpenAPI; release notes; acta de aceptación; tag y plan de rollback.

Riesgos críticos a validar

- **Seguridad:** JWT/refresh, autorización por gymId, secretos fuera de Git, validación de entradas, CORS, webhook firmado, auditoría de acceso.

- **Datos:** transacciones para pagos/reservas, migraciones Flyway, integridad e idempotencia, copia y restauración medibles.

- **IA:** diferenciar reglas deterministas de un modelo de IA, documentar fallback, privacidad, límites y revisión de recomendaciones.

- **Experiencia:** estados vacíos/error, móvil, contraste, teclado, lectores de pantalla y ayuda contextual.

Release y reversión

Congelar release; ejecutar CI y E2E; revisar migraciones en copia de datos; desplegar a entorno de prueba; aceptación; etiquetar main; desplegar; monitorear. Si falla, revertir aplicación y aplicar procedimiento probado de datos; nunca editar migraciones ya aplicadas.

9. Ajustes necesarios antes de fijar el plan

Este apartado separa hechos observados de decisiones pendientes. Evita que el documento presente tareas antiguas como trabajo nuevo o porcentajes calculados a partir de estados desactualizados.

Hallazgo	Acción y responsable propuesto
Jira: 303/303 'por hacer'; responsables asignados y 0 vencimientos.	PPP-301: auditar contra Git y actualizar estados con evidencia; Alejandro.
Jira: sprints de abril superpuestos y sobrecargados.	PPP-302: reestimar con capacidad real y acordar fecha final; Alejandro.
Notion antiguo: Node.js/NestJS; repositorio: Spring Boot/JDBC.	PPP-303: actualizar arquitectura y criterios técnicos; Juan Diego.
PPP-21 pagos estaba bajo épica de guías.	Corregido: ahora depende de PPP-3, épica de pagos.
No se observa módulo de hábitos ni interfaz de voz en runtime.	Confirmar prioridad/alcance y crear criterios verificables; Juan José + equipo.
IA actual usa reglas deterministas; requisitos hablan de IA personalizada.	Definir si MVP por reglas cumple o si se exige modelo; medir y documentar.
Tests activos: smoke y contrato; faltan suites unitarias en frontend/backend.	Añadir pruebas de negocio y seguridad antes de release; Alejandro coordina.

Primeras diez acciones

1) Validar el decisor de producto. 2) Aprobar alcance y prioridades. 3) Ejecutar PPP-301: conciliar Jira con Git. 4) Revisar la rama de migración. 5) Ejecutar PPP-302: reestimar sprints 9-12. 6) Ejecutar PPP-304: protecciones y plantilla de PR. 7) Fijar Definition of Ready/Done. 8) Crear pruebas críticas. 9) Ejecutar PPP-303: actualizar arquitectura. 10) Publicar línea base y fecha aprobada.

10. Fuentes, supuestos y control

Fuentes consultadas el 22 de octubre de 2026. Los enlaces permiten verificar alcance, fechas y decisiones; el repositorio es evidencia del runtime, mientras Jira y Notion contienen intención y planeación.

Notion

- Página principal de Power Zone: propósito y acceso a documentos.
- Especificación de requisitos: 14 módulos, 40 requisitos funcionales y 22 no funcionales; contiene referencias técnicas históricas.
- Informe general del sistema: objetivos y alcance.
- Planteamiento del problema: problema y roles históricos.
- Acta de planeación Sprint 1 documental: evidencia de trabajo de documentación en 2025; no es el Sprint 1 de Jira 2026.
- Plan de desarrollo en Notion: responsabilidades aplicadas, reglas y riesgos de cronograma.

Jira y repositorio

- Backlog PPP en Jira: 16 épicas, 40 historias, 243 subtareas, 4 tareas transversales y sprints.
- PPP-21: historia de pagos corregida bajo PPP-3.
- Repositorio GitHub: README, docs/ARCHITECTURE.md, .github/workflows/ci.yml y commits consultados localmente.

Control de cambios	Los roles propuestos ya se aplicaron en Jira y se documentaron en Notion. Las fechas de Jira siguen pendientes de ajuste por capacidad. El porcentaje 0-100 es una estructura de hitos y no mide avance real; no se cambiaron estados ni ramas.

Próxima revisión: estimar carga, confirmar alcance del MVP y aprobar la nueva fecha final. Registrar la decisión en Jira y Notion.